



boolean

DOM



JavaScript

Today, JavaScript is **everywhere**! You use it to program things from **toasters** to **spaceship apps**, **frontends**, **backends** and pretty much anything you can think of

However, it is still the only language that is natively run by browsers, and so, it allows you to do some unique things, like:

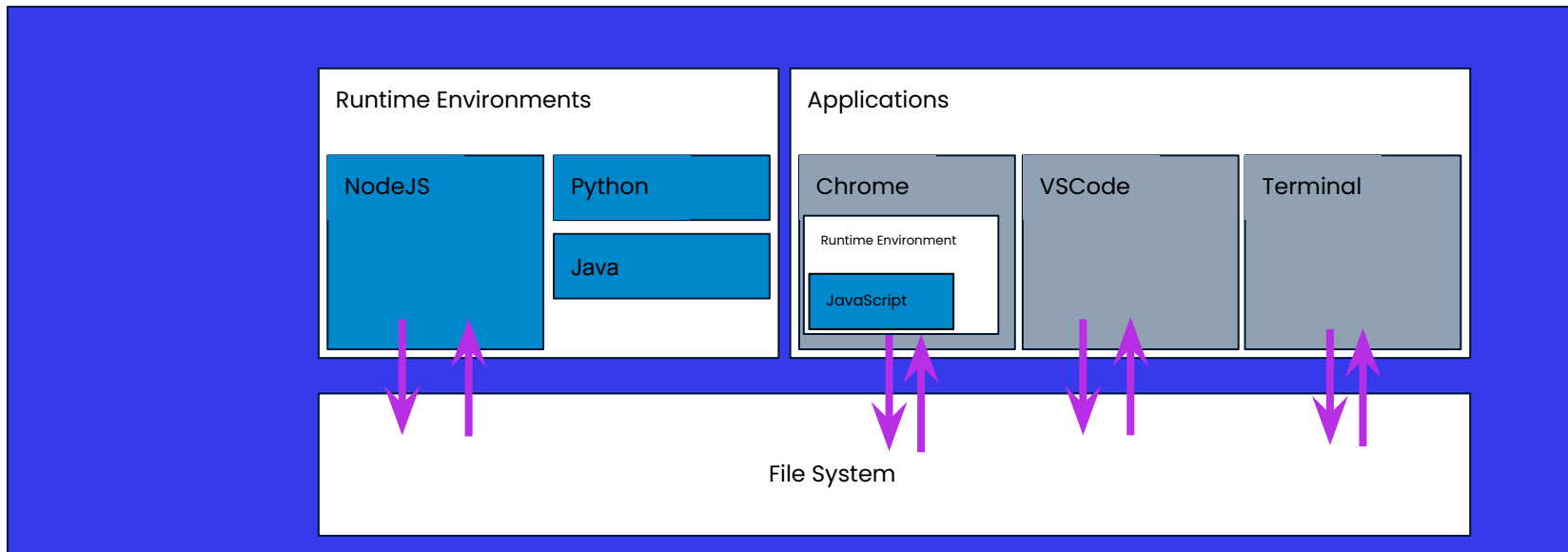
- Create, read, update and delete **elements** on the page
- React to user **events** (clicks, scrolls, etc)
- Validate **data** entered by the user
- Check which browser is being used
- Check the current size of the screen
- And much more...

These features allow us to create **entire applications** within the browser.



JavaScript (Which one?!)

Your computer (simplified)





Nodejs !== JS in the browser

Treat Nodejs and JavaScript as entirely separate programming languages – like Python and Java, but ones that share almost identical syntax.

To run a Nodejs program, you would typically use your command line to instruct your **computer** to run the NodeJS program with a JS file.

```
→ boolean node index.js
```

To run a JS program, you would typically use your **browser** to load an HTML page which loads a JavaScript file.



Livecoding

- **Your teacher will demonstrate setting up a project for local development using html, css, javascript.**
- **Your teacher will demonstrate using the browser directly, and using VSCode's LiveServer**



Time for some practice

- Create a new project directory
- Create a HTML file and a JS file
- Use a script element to link the JS file to your HTML file
- In the JS file, log any string the console.
- Use your browser directly and then use LiveServer to load the HTML page
- Open up the browser console and verify you see your logs!



The DOM (Document Object Model)



In this section, we're going to be looking at what happens in the browser once it receives the HTML we've written, and how we can use JavaScript to change what's on the page in real time, upgrading our static sites into **dynamic applications!**



**What do we mean
by *static* and
dynamic?**



The DOM (Document Object Model)

```
<body>
  <h1>Hello!</h1>
  <div class="card">
    <img src="" alt="" />
    <h2>Card title</h2>
    <p>Card description</p>
  </div>
</body>
```

When you are writing **HTML**, your file is **one big text document**, which means your entire code is basically just **one big string**!



The DOM (Document Object Model)

```
<body>
```

```
<h1>Hello!</h1>
```

```
<div class="card">
```

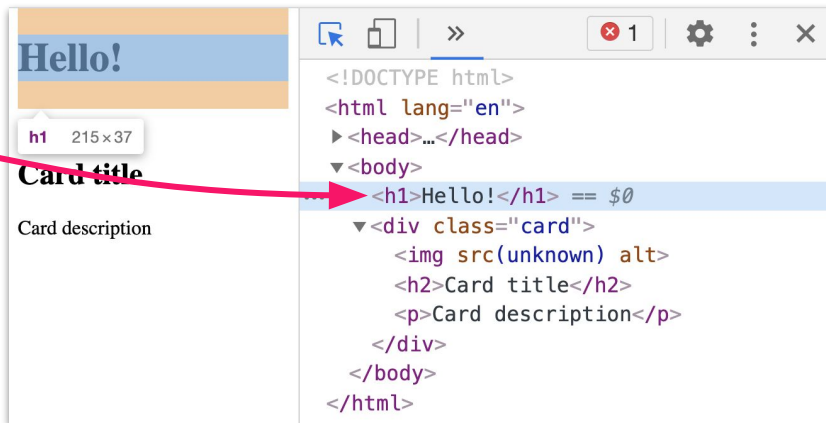
```
<img src="" alt="" />
```

```
<h2>Card title</h2>
```

```
<p>Card description</p>
```

```
</div>
```

```
</body>
```

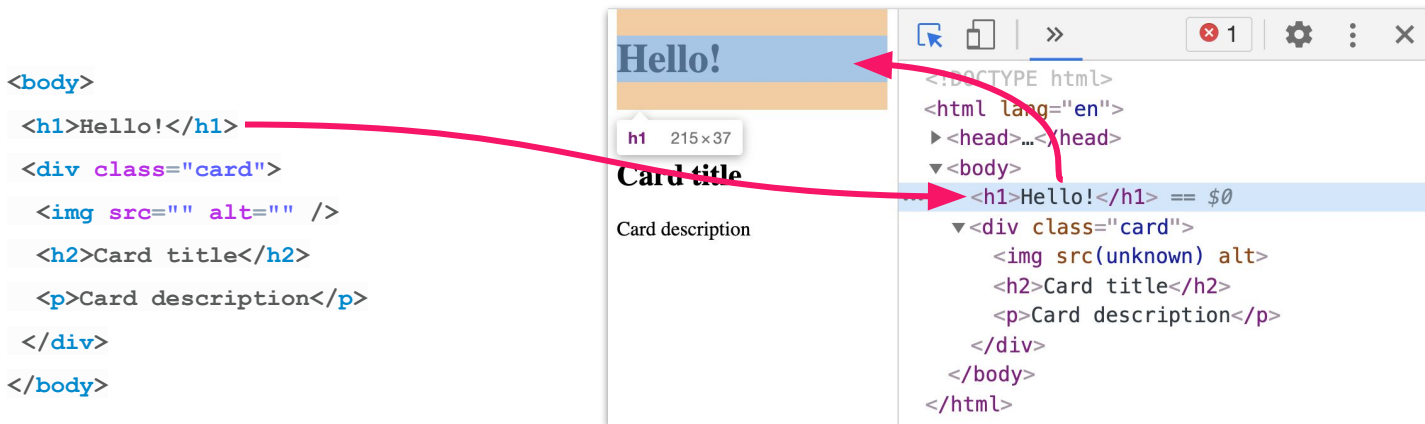


When your browser sees that big string, it **parses** it and turns it into a **tree of elements**

This representation of the page is called the **DOM**, or **Document Object Model**



The DOM (Document Object Model)



Once the **elements** are created, the browser **paints them** on the **page**

You can see how the **H1 element** on the right is directly linked to the painted **Hello!** on the page

You can even edit elements on the right in real time and see them change on the page, **without having to refresh**. Head over to a popular page and give it a try!



DOM CRUD

Just like with **arrays** and **objects**, **elements** on the page are also a **resource** that we can manage. Meaning, we can **create**, **read**, **update** and **delete** elements, too!

JavaScript is a unique language in that it is the **only programming language** that **browsers understand**

Because of this, **our code** has privileged access to **the page**, and we can use it to directly manipulate elements on the page in real time, **without having to refresh** the page

It all starts with a little object that the browser gives us, called the **document**. Type it in the console and explore it a little!



Livecoding

- **Your teacher will demonstrate each of the following DOM operations**
- **Create at least 1 element**
- **Read at least 1 element**
- **Update at least 1 element**
- **Delete at least 1 element**



DOM CRUD

Creating here means **creating an element**. The **document** has a **method** we can use called **createElement**

```
const h1El = document.createElement('h1')
```

At this point, the element **exists**, but it's actually **not on the page yet!** To add it to the page, we need to find an element that's already on the page, and make our element its child by **appending it**

```
document.body.append(h1El)
```

Because the **body element** is such a basic part of our html, our document gives us quick access to it with **document.body**. All elements have an **append()** method that takes the element we'd like to append



DOM CRUD

Reading means **finding an element on the page**. Once again, the **document** object has a useful method for us: **querySelector**

```
const divEl = document.querySelector('div')
```

That string that we pass as an argument can be any valid CSS selector! So we can do things like `'.class'`, `'#id'`, `'div #id-inside-a-div'`, and all the greatest hits

One thing to note is that **querySelector** finds only the **first matching element**. If you need get **all matches**, just use **querySelectorAll** instead!

```
const allDivs = document.querySelectorAll('div')
```



DOM CRUD

Updating means **changing anything** about our element. We can **add/replace text**:

```
const h1El = document.querySelector('h1') // <h1></h1>
h1El.innerText = "Welcome to my page!" // <h1>Welcome to my page!</h1>
```

Add/remove any **attribute**:

```
h1El.setAttribute('class', 'my-title') // <h1 class="my-title">Welcome to my page!</h1>
h1El.removeAttribute('class') // <h1>Welcome to my page!</h1>
```

Or even use convenient syntax to **add inline styling** and other useful things:

```
h1El.style.color = "red" // <h1 style="color: red;">Welcome to my page!</h1>
```

If the element is already on the page, you will see your changes **right away**!



DOM CRUD

Deleting simply means removing an element from the page. Elements have a `remove()` method:

```
const h1El = document.querySelector('h1')  
h1El.remove()
```

That will remove the element from the page, along with any children. Worth noting that the element is not completely gone, since you still have it in your `h1El` variable and could easily re-attach it to the page!

```
document.body.append(h1El)
```



Time for some practice

- Go to your favourite page
- Using only the console:
 - Create at least 1 element
 - Read at least 1 element
 - Update at least 1 element
 - Delete at least 1 element
- Give it a personal touch and share a screenshot! (fake news?!)

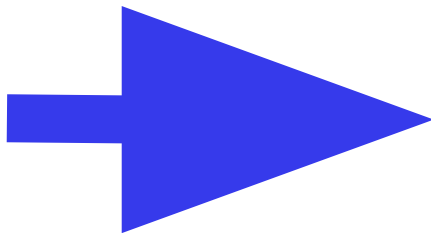


Simple use case:

JavaScript has your data and you want to instruct the browser to show it. A typical scenario might be that your data are in an array and you want to show it on the page.

1. Loop through your array.
2. For each item, create an element and append it an element that's currently on the page

```
const fizzbuzz = () => {  
  // returns an array of values according to the rules  
  of fizzbuzz  
}  
  
const body = document.querySelector("body")  
const result = fizzbuzz()  
  
result.forEach(value => {  
  const p = document.createElement("p")  
  p.innerText = value  
  body.append(p)  
})
```



```
<body>  
<p>fizzbuzz</p>  
<p>1</p>  
<p>2</p>  
<p>fizz</p>  
<p>4</p>  
<p>buzz</p>  
<p>fizz</p>  
<p>7</p>  
<p>8</p>  
<p>fizz</p>  
<p>buzz</p>  
<p>11</p>  
<p>fizz</p>  
<p>4</p>  
<p>4</p>  
<p>fizzbuzz</p>  
</body>
```



Livecoding

- **Your teacher will demonstrate writing fizzbuzz that shows results on a webpage**



Time for some practice

- **Your turn!**
- **Store the months of the year in an array.**
- **Loop through the array and show each item on the webpage**
- **Extension: Give it a personal touch and use JavaScript to add classnames to your elements, and CSS to style each element so you have a set of cards for each month in the center of the page, in a 3x4 grid.**